

İSTANBUL BEYKENT ÜNİVERSİTESİ

Bilgisayar Mühendisliği (TR) Bölümü

Gelişmiş Veri Tabanı Uygulamaları

FintechDB – Fintech Sektörü Veritabanı Sistemi

DÖNEM SONU PROJE RAPORU

Öğrenci Adı Soyadı	Öğrenci Numarası
EMİR KARAMAN	2303022018
METEHAN ŞENER	2303022052
SUDE NUR KARABULUT	2503022072

Teslim Tarihi: 1 Haziran 2026

1. PROJE KONUSU VE İŞ SENARYOSU

FintechDB, fintech (finansal teknoloji) sektörüne yönelik geliştirilmiş bir SQL Server veritabanı projesidir. Proje; müşteri yönetimi, hesap işlemleri, kredi başvurusu değerlendirmesi ve gerçek zamanlı dolandırıcılık tespiti gibi temel bankacılık süreçlerini modellemektedir.

Sistem tasarımında aşağıdaki iş gereksinimleri kullanılmıştır:

- Müşterilerin T.C kimlik benzersizliği ve e- posta doğrulaması ile doğru şekilde kaydedilmesi.
- Her müşterinin birden fazla hesaba (vadesiz, vadeli, kredi) sahip olabilmesi
- HAVALE, EFT, ATM, POS gibi farklı tiplerde finansal işlem kaydı
- Kredi başvurusu için akış: başvuru alımı, değerlendirme, sonuçlandırma
- Yüksek tutarlı (>50.000 birim) işlemlerin otomatik olarak fraud şüphelisi olarak tutulması
- Makine öğrenmesi modeli (Lojistik Regresyon) ile kredi onay tahmini

2. ER DİYAGRAMI VE İLİŞKİ AÇIKLAMASI

Veritabanı toplamda 7 ana tablo içermektedir. Tablolar ve aralarındaki birincil anahtar (PK) / yabancı anahtar (FK) ilişkileri aşağıdaki gibidir:

Tablo	PK	FK	Açıklama
Musteri	MusteriID	—	Müşteri ana kaydı
Hesap	HesapID	MusteriID → Musteri	Banka hesabı
Islem	IslemID	HesapID → Hesap	Finansal işlem kaydı
Kredi_Basvuru	BasvuruID	MusteriID → Musteri	Kredi başvurusu
Kredi_Skoru	SkorID	MusteriID → Musteri	T-SQL bazlı skor
Fraud_Log	LogID	IslemID → Islem	Fraud uyarı kaydı
ML_Model_Sonuc	SonucID	MusteriID, BasvuruID	ML tahmin çıktısı

İLİŞKİ TÜRLERİ

- Musteri – Hesap: Bire-çok (1:N). Bir müşterinin birden fazla hesabı olabilir.
- Hesap – İşlem: Bire-çok (1:N). Bir hesap altında birden fazla işlem kaydedilebilir.
- Musteri – Kredi_Basvuru: Bire-çok (1:N). Bir müşteri birden fazla kredi başvurusu yapabilir.
- Musteri – Kredi_Skoru: Bire-çok (1:N). Farklı yöntemlerle hesaplanan skorlar saklanır.
- İşlem – Fraud_Log: Bire-bir (1:1). Her fraud kaydı tek bir işleme bağlıdır.
- Musteri + Kredi_Basvuru – ML_Model_Sonuc: ML çıktısı hem müşteriyle hem başvuruyla ilişkilendirilir.

3. NORMALİZASYON BELGESİ

Her tablo için fonksiyonel bağımlılık analizi yapılmış ve 3NF koşulunun sağlandığı doğrulanmıştır.

1NF (1.NORMAL FORM)

Kural: Her sütun atomik değer içermeli, tekrar eden grup olmamalı, her satır birincil anahtarla benzersiz tanımlanmalıdır.

Tüm tablolarda her sütun bölünemez tek bir değer taşımaktadır. Müşteri adı ve soyadı ayrı sütunlarda tutulmuş, birleşik alan kullanılmamıştır. Bir müşterinin birden fazla hesabı olması durumunda hesaplar aynı satıra yığılmamış, Hesap adında ayrı bir tablo açılarak her kayıt kendi satırında saklanmıştır. Her tabloda INT IDENTITY tipinde birincil anahtar tanımlandığından her satır benzersiz şekilde tanımlanmaktadır.

2NF (2.NORMAL FORM)

Kural: 1NF sağlanmış olmalı, anahtar olmayan hiçbir sütun bileşik birincil anahtarın yalnızca bir parçasına bağlı olmamalıdır.

Tüm tablolarda birincil anahtar tek sütunludur (MusteriID, HesapID, İşlemID vb.). Kısmi bağımlılık yalnızca bileşik PK'larda oluşabilir; bu projede böyle bir yapı bulunmadığından tüm tablolar 1NF'den doğrudan 2NF'e geçmektedir.

3NF (3.NORMAL FORM)

Kural: 2NF sağlanmış olmalı, anahtar olmayan hiçbir sütun başka bir anahtar olmayan sütuna bağlı olmamalıdır.

Her tablo için potansiyel geçişli bağımlılıklar tek tek incelenmiştir:

Musteri tablosunda TCKN sütünü benzersiz bir değer taşısa da birincil anahtar olarak kullanılmamıştır. Diğer sütunlar (Ad, Soyad, Email) TCKN'ye değil doğrudan MusteriID'ye bağlıdır; bu nedenle geçişli bağımlılık yoktur.

Islem tablosunda FraudFlag sütununun Tutar sütununa bağlı olup olmadığına bakılmıştır. Tutar > 50.000 TL koşulunu değerlendiren mantık tabloya gömülü bir bağımlılık değil, trigger tarafından uygulanan dış bir iş kuralıdır. Tabloda FraudFlag, IslemID'ye doğrudan bağlı bağımsız bir sütundur.

ML_Model_Sonuc tablosunda Cikti (0–1 arası olasılık değeri) ile TahminEtiketi (ONAY/RED) arasındaki ilişki en dikkatli değerlendirilen noktadır. Eşik değeri (0.5) uygulama kodunda belirlendiğinden Cikti tek başına TahminEtiketi'ni fonksiyonel olarak belirlememektedir. Her iki sütun birbirinden ayrı tutulmuştur.

Kredi_Basvuru tablosunda TalepEdilenTutar, Vade ve AylıkGelir sütunlarının birbirini belirleyip belirlemediği incelenmiştir. Bu üç sütun birbirinden bağımsızdır; aynı müşteri farklı başvurularda farklı kombinasyonlar oluşturabilir.

Diğer tablolarda (Hesap, Kredi_Skoru, Fraud_Log) geçişli bağımlılık yoktur.

Sonuç: 7 tablonun tamamı 3NF koşulunu sağlamaktadır.

Tablo	Fonksiyonel Bağımlılıklar	3NF İhlali Var mı?
Musteri	MusteriID → Ad, Soyad, TCKN, Email...	Hayır
Hesap	HesapID → MusteriID, Bakiye, Tip...	Hayır
Islem	IslemID → HesapID, Tutar, Tip...	Hayır
Kredi_Basvuru	BasvuruID → MusteriID, Tutar, Vade...	Hayır
Kredi_Skoru	SkorID → MusteriID, Skor, Yöntem...	Hayır
Fraud_Log	LogID → IslemID, RiskSkoru, Açıklama	Hayır
ML_Model_Sonuc	SonucID → MusteriID, BasvuruID, Çıktı...	Hayır

4. TABLO TANIMLARI

Musteri

Sütun	Veri Tipi	Kısıt	Açıklama
MusteriID	INT IDENTITY	PK	Otomatik artan birincil anahtar
Ad	NVARCHAR(50)	NOT NULL	Müşteri adı
Soyad	NVARCHAR(50)	NOT NULL	Müşteri soyadı
TCKN	CHAR(11)	UNIQUE, CHECK	T.C. Kimlik No – tam 11 karakter zorunlu
DogumTarihi	DATE	NULL	Doğum tarihi (NULL ise boş bırakılabilir)
Email	NVARCHAR(100)	UNIQUE	Benzersiz e-posta adresi
Telefon	NVARCHAR(15)	NULL	Müşteri telefon numarası (NULL ise boş)
KayıtTarihi	DATETIME	DEFAULT GETDATE()	Kayıt oluşturulma tarihi

Hesap

Sütun	Veri Tipi	Kısıt	Açıklama
HesapID	INT IDENTITY	PK	Otomatik artan PK
HesapNo	VARCHAR(20)	UNIQUE, NOT NULL	Benzersiz hesap numarası
MusteriID	INT	FK, NOT NULL	Hesabın sahibi müşteri
Bakiye	DECIMAL(18,2)	CHECK ≥ 0	Anlık bakiye (negatif olamaz)
ParaBirimi	CHAR(3)	CHECK IN(TRY,USD,EUR)	Para birimi kodu
HesapTipi	NVARCHAR(20)	CHECK	VADESİZ / VADELİ / KREDİ
Durum	NVARCHAR(20)	DEFAULT AKTIF	AKTIF / PASİF / KAPALI
AcilisTarihi	DATETIME	DEFAULT GETDATE()	Hesabın açılış tarihi

İslem

Sütun	Veri Tipi	Kısıt	Açıklama
İslemID	INT IDENTITY	PK	Otomatik artan birincil anahtar
HesapID	INT	FK, NOT NULL	İşlemin yapıldığı hesap
Tutar	DECIMAL(18,2)	CHECK > 0	İşlem tutarı (sıfır veya negatif olamaz)
İslemTipi	NVARCHAR(20)	CHECK	HAVALE / EFT / ATM / POS
İslemTarihi	DATETIME	DEFAULT GETDATE()	İşlemin gerçekleşme zaman damgası
Acıklama	NVARCHAR(200)	NULL	İşlem açıklaması (isteğe bağlı)
FraudFlag	BIT	DEFAULT 0	Fraud şüphesi: 0=Normal, 1=Şüpheli (trigger tarafından set edilir)

Kredi_Basvuru

Sütun	Veri Tipi	Kısıt	Açıklama
BasvuruID	INT IDENTITY	PK	Otomatik artan birincil anahtar
MusteriID	INT	FK, NOT NULL	Başvuruyu yapan müşteri
BasvuruTarihi	DATETIME	DEFAULT GETDATE()	Başvurunun yapıldığı tarih
TalepEdilenTutar	DECIMAL(18,2)	CHECK > 0	İstenilen kredi miktarı
Vade	INT	CHECK > 0	Geri ödeme süresi (ay)
AylıkGelir	DECIMAL(18,2)	CHECK >= 0	Müşterinin beyan ettiği aylık gelir
Durum	NVARCHAR(20)	DEFAULT/CHECK	BEKLEMEDE (default) / ONAYLANDI / REDDEDİLDİ
DegerlendirmeTarihi	DATETIME	NULL	Trigger tarafından otomatik doldurulur (GETDATE())

Kredi_Skoru

Sütun	Veri Tipi	Kısıt	Açıklama
SkorID	INT IDENTITY	PK	Otomatik artan birincil anahtar
MusteriID	INT	FK, NOT NULL	Skoru hesaplanan müşteri
Skor	DECIMAL(5,4)	CHECK 0–1	Kredi skoru (0.0000 – 1.0000 aralığında)
HesaplamaTarihi	DATETIME	DEFAULT GETDATE()	Skorun hesaplanma tarihi
HesaplamaYontemi	NVARCHAR(50)	NOT NULL	Ör. "SADE_LINEER" – hangi algoritmanın kullanıldığını gösterir

Fraud_Log

Sütun	Veri Tipi	Kısıt	Açıklama
LogID	INT IDENTITY	PK	Otomatik artan birincil anahtar
IslemID	INT	FK, NOT NULL	Fraud şüphesi taşıyan işlem kaydı
RiskSkoru	DECIMAL(5,4)	CHECK 0–1	Risk olasılığı; trigger tarafından 0.90 olarak set edilir
Aciklama	NVARCHAR(500)	NULL	İsteğe bağlı açıklama metni
LogTarihi	DATETIME	DEFAULT GETDATE()	Log kaydının oluşturulma tarihi

ML_Model_Sonuc

Sütun	Veri Tipi	Kısıt	Açıklama
SonucID	INT IDENTITY	PK	Otomatik artan birincil anahtar
MusteriID	INT	FK, NOT NULL	Tahminin yapıldığı müşteri
BasvuruID	INT	FK, NOT NULL	Değerlendirilen kredi başvurusu
ModelAdi	NVARCHAR(50)	NOT NULL	Kullanılan ML model adı (ör. LogisticRegression)
Cikti	DECIMAL(5,4)	CHECK 0–1	Modelin onay olasılığı (0.0000–1.0000)
TahminEtiketi	NVARCHAR(20)	CHECK	“ONAY” veya “RED” – modelin insan okunur kararı
Tarih	DATETIME	DEFAULT GETDATE()	Tahmin zaman damgası

5. SQL BİLEŞENLERİNİN AÇIKLANMASI

VIEW Tanımları

View Adı	İçerik	Amaç
v_aktif_hesaplar	SELECT – WHERE Durum='AKTIF'	Pasif/Kapalı hesapları gizleyerek sade erişim
v_kredi_raporu	INNER JOIN Musteri + Kredi_Basvuru	Ad Soyad ile birleşik kredi başvurusu raporu
v_fraud_ozet	COUNT(*) + SUM(FraudFlag) – GROUP BY	Hesap bazında toplam ve şüpheli işlem sayısı

Örnek kullanım (v_kredi_raporu):

```
SELECT * FROM dbo.v_kredi_raporu WHERE Durum = 'BEKLEMEDE';
```

	BasvuruID	Musteri	TalepEdilenTutar	Vade	Durum
1	6	Emir Çorlu	80000.00	24	BEKLEMEDE
2	9	Sıdar Bilgin	10000.00	6	BEKLEMEDE
3	12	Buse Gül	35000.00	12	BEKLEMEDE
4	14	Muhammet Yaman	120000.00	36	BEKLEMEDE

Çıktı: Henüz değerlendirilmemiş kredi başvurularını müşteri adı ile listeler.

Stored Procedure Tanımları

HesaplaKrediSkoru (@MusteriID INT)

Parametrelili SELECT prosedürüdür. Müşterinin toplam hesap bakiyesi ve son kredi başvurusundaki beyan ettiği aylık geliri baz alarak basit doğrusal bir skor hesaplar ve Kredi_Skoru tablosuna yazar.

```
EXEC dbo.HesaplaKrediSkoru @MusteriID = 1;
```

	SkorID	MusteriID	Skor	HesaplamaTarihi	HesaplamaYontemi
1	1	1	1.0000	2026-05-26 17:51:36.593	SADE_LINEER

Çıktı: Kredi_Skoru tablosuna yeni bir satır eklenir (Skor alanı 0–1 aralığında).

IslemEkle (@HesapID, @Tutar, @IslemTipi)

İşlem tipine göre bakiye günceller (POS/EFT/HAVALE için düşer, ATM için artırır). Yetersiz bakiye durumunda hata fırlatır ve işlem kaydı oluşturmaz.

```
EXEC dbo.IslemEkle @HesapID=1, @Tutar=500, @IslemTipi='POS';
```

	HesapID	MusteriID	HesapNo	Bakiye	ParaBirimi	HesapTipi	AcilisTarihi	Durum
1	1	1	TR00100001	24500.00	TRY	VADESIZ	2026-05-26 18:06:30.870	AKTIF
2	2	1	TR00100002	35000.00	TRY	VADELI	2026-05-26 18:06:30.870	AKTIF

Çıktı: HesapID=1'in bakiyesini sorgular (25.000 TL). 500 TL <25.000 TL olduğundan işlem geçer. Hesap tablosundaki bakiye 24.500 TL'ye güncellenir.

16	16	1	500.00	POS	2026-05-26 18:06:45.923	NULL	0
----	----	---	--------	-----	-------------------------	------	---

Ardından Islem tablosuna yeni satır eklenir.

Hata senaryosu (yetersiz bakiye):

```
EXEC dbo.IslemEkle @HesapID=1, @Tutar=99000, @IslemTipi='HAVALE';
```


İletiler

İleti 50000, Düzey 16, Durum 1, Yordam dbo.IslemEkle, Satır 16 [Toplu İş Başlangıç Satırı 0]
Yetersiz bakiye! İşlem iptal edildi.

Tamamlanma süresi:2026-05-26T19:03:13.8680713+03:00

Çıktı: Bakiye 24.500 TL, talep 99.000 TL olduğundan prosedür RAISERROR fırlatır ne bakiye güncellenir ne de İşlem satırı oluşur.

Trigger Tanımları

trg_FraudKontrol (AFTER INSERT – İşlem)

Loglama triggeridir. İşlem tablosuna yeni bir kayıt eklendiğinde tutar 50.000 TL'yi aşıyorsa FraudFlag alanını otomatik 1 olarak günceller ve Fraud_Log tablosuna kayıt ekler (RiskSkoru = 0.90).

```
INSERT INTO dbo.Islem (HesapID, Tutar, IslemTipi, Aciklama)
VALUES (7, 75000, 'EFT', 'Araba Alımı');
```

18	18	7	75000.00	EFT	2026-05-26 18:13:53.333	Araba Alımı	1
----	----	---	----------	-----	-------------------------	-------------	---

Çıktı: Eklenen satırın FraudFlag sütunu 0'dan 1'e güncellenir.

Sonuçlarİletiler

	LogID	IslemID	RiskSkoru	Aciklama	LogTarihi
1	1	18	0.9000	Yüksek tutarlı işlem tespiti	2026-05-26 18:13:53.353

Fraud_Log tablosuna kayıt yazılır.

trg_BasvuruDegerlendirme (AFTER UPDATE – Kredi_Basvuru)

İş kuralı triggeridir. Kredi başvurusunun Durum sütunu değiştiğinde (örn. BEKLEMEDE → ONAYLANDI) DegerlendirmeTarihi alanını otomatik olarak GETDATE() ile doldurur.

```
UPDATE dbo.Kredi_Basvuru
SET Durum = 'ONAYLANDI'
WHERE BasvuruID = 6;
```

Güncelleme öncesi:

6	6	6	2026-05-06 00:00:00.000	80000.00	24	35000.00	BEKLEMEDE	NULL
---	---	---	-------------------------	----------	----	----------	-----------	------

Güncelleme sonrası:

6	6	6	2026-05-06 00:00:00.000	80000.00	24	35000.00	ONAYLANDI	2026-05-26 18:23:59.810
---	---	---	-------------------------	----------	----	----------	-----------	-------------------------

Index Tanımları

Index Adı	Tablo	Tür	Sütunlar
IX_Islem_HesapID_Tarih	Islem	Composite	HesapID, IslemTarihi
IX_Islem_HesapID	Islem	Non-clustered	HesapID
IX_Hesap_MusteriID	Hesap	Non-clustered	MusteriID
IX_Kredi_MusteriID	Kredi_Basvuru	Non-clustered	MusteriID

6. YETKİLENDİRME TASARIMI

Rol Tanımları

Rol	Kullanıcı	Açıklama
db_readonly	fintech_reader	Yalnızca okuma; tüm tablolar ve view'lara SELECT yetkisi
db_operator	fintech_operator	SELECT + INSERT + UPDATE; IslemEkle SP çalıştırabilir; Musteri tablosuna DELETE yetkisi REVOKE edilmiştir

Yetki Matrisi

Nesne	SELECT	INSERT	UPDATE	DELETE
Musteri (reader)	✓	—	—	—
Hesap (operator)	✓	✓	✓	—
Islem (operator)	✓	✓	✓	—
Musteri (operator) – DELETE	✓	—	—	REVOKE
v_aktif_hesaplar (reader)	✓	—	—	—
IslemEkle SP (operator)	EXECUTE	—	—	—

REVOKE senaryosu: fintech_operator rolüne başlangıçta DELETE yetkisi verilmiş, ardından güvenlik politikası gereği bu yetki geri alınmıştır. Böylece müşteri silme işlemi yalnızca dbo (veritabanı sahibi) tarafından yapılabilmektedir.

7. VERİ AMBARI ŞEMASI VE OLAP SORGULARI

Yıldız Şema (Star Schema)

Veri ambarı katmanı, dbo şemasından ayrı olarak Fact_Islem isimli bir olgu tablosu ve üç boyut tablosundan oluşmaktadır:

Tablo	Tür	İçerik
Fact_Islem	Fact	IslemID, MusteriID, HesapID, TarihID, Tutar, FraudFlag
Dim_Musteri	Boyut	MusteriID, AdSoyad, TCKN, Email
Dim_Hesap	Boyut	HesapID, HesapNo, HesapTipi, ParaBirimi
Dim_Tarih	Boyut	TarihID (YYYYMMDD), Yil, Ay, Gun

ETL Mantığı

Kaynak tablolardan (OLTP) hedef tablolara (OLAP) aktarım INSERT...SELECT yöntemiyle gerçekleştirilmiştir. Tarih boyutu için surrogate key olarak YEAR*10000+MONTH*100+DAY formülü kullanılmıştır.

```
INSERT INTO dbo.Fact_Islem (IslemID, MusteriID, HesapID, TarihID, Tutar,
FraudFlag)
SELECT i.IslemID, h.MusteriID, i.HesapID,
YEAR(i.IslemTarihi)*10000+MONTH(i.IslemTarihi)*100+DAY(i.IslemTarihi),
i.Tutar, i.FraudFlag
FROM dbo.Islem i INNER JOIN dbo.Hesap h ON i.HesapID = h.HesapID;
```

OLAP Sorguları

ROLLUP – Yıl/Ay bazlı toplam ciro

Yıl ve ay gruplandırmasında ara toplam ve genel toplam üretir.

```
SELECT dt.Yil, dt.Ay, SUM(f.Tutar) AS ToplamCiro
FROM dbo.Fact_Islem f INNER JOIN dbo.Dim_Tarih dt ON f.TarihID = dt.TarihID
GROUP BY ROLLUP(dt.Yil, dt.Ay);
```

CUBE – HesapTipi x ParaBirimi çapraz analiz

Tüm kombinasyonlar için alt toplam ve genel toplam üretir.

```
SELECT dh.HesapTipi, dh.ParaBirimi, SUM(f.Tutar) AS ToplamHacim
FROM dbo.Fact_Islem f INNER JOIN dbo.Dim_Hesap dh ON dh.HesapID = f.HesapID
GROUP BY CUBE(dh.HesapTipi, dh.ParaBirimi);
```

8. EŞZAMANLILIK VE TRANSACTION YÖNETİMİ

Transaction Senaryoları

Senaryo	Açıklama	Beklenen Sonuç
1 – Para Transferi	Hesap 1'den 500 TL düşülür, Hesap 3'e eklenir; iki Islem kaydı oluşturulur	COMMIT – tüm işlemler atomik
2 – Kredi Kullandırım	Başvuru ONAYLANDI olarak güncellenir, hesaba kredi tutarı yüklenir	COMMIT – kredi ve bakiye güncelleme atomik

3 – CHECK İhlali	Bakiyeyi eksi yapacak UPDATE denemesi yapılır	ROLLBACK – CHECK constraint hatası yakalanır
-------------------------	---	--

Isolation Level Seçimi ve Gerekçesi

Projede READ COMMITTED isolation level benimsenmiştir (SQL Server varsayılanı). Bu kararın gerekçeleri:

- Kirli okuma (dirty read) engellenir: onaylanmamış bakiye güncellemeleri diğer oturumlar tarafından görülmez.
- Tekrar edilemeyen okuma (non-repeatable read) riski kabul edilebilir seviyededir; fintech demo ortamında bu düzey yeterlidir.
- SERIALIZABLE seçimi deadlock riskini artıracığından tercih edilmemiştir.
- Tüm kritik işlemler (para transferi, kredi kullandırım) BEGIN TRANSACTION / COMMIT / ROLLBACK bloğu içinde yönetilmekte olup TRY...CATCH ile hata senaryoları güvenle ele alınmaktadır.

9. BİREYSEL KATKI TABLOSU

Grup Üyesi	Geliştirilen Bileşenler
Sude Nur Karabulut	DDL şema tasarımı (01_ddl.sql), DML örnek verileri (02_dml.sql), ER diyagramı (DBML + PNG), normalizasyon analizi, proje raporu
Emir Karaman	ML pipeline (train_model.py, predict_and_save.py, generate_dml.py), Stored Procedure tasarımı (05_procedures.sql), Veri ambarı (09_datawarehouse.sql)
Metehan Şener	Sorgular (03_queries.sql), VIEW tanımları (04_views.sql), Trigger tasarımı (06_triggers.sql), Transaction yönetimi (07_transactions.sql), Güvenlik ve yetkilendirme (08_security.sql)

10. KAYNAKÇA

- Microsoft Docs (2024). SQL Server Transact-SQL Reference. <https://learn.microsoft.com/tr-tr/sql/t-sql/>
- Microsoft Docs (2024). Database Engine Tuning Advisor & Index Design Guidelines. <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/>
- Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit (3rd ed.). Wiley. (Star schema ve ETL tasarım prensipleri)
- scikit-learn developers (2024). LogisticRegression – API Reference. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- OWASP (2023). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html